

FinAgent: A Fault-Tolerant, Multi-Agent Orchestrator for Real-Time Financial Analysis

Hackathon Submission

July 2026

Abstract

FinAgent is a real-time, multi-agent financial orchestration system designed to process disparate market signals and unstructured documents into structured, highly observable trading decisions. Powered by the 120-billion parameter `openai/gpt-oss-120b` language model on Groq, the system integrates a Retrieval-Augmented Generation (RAG) pipeline with a deterministic LangGraph state machine. This report details the system architecture, dual-agent design, routing logic, RAG implementation, and the comprehensive observability features implemented to ensure strict financial compliance and multi-level fault tolerance.

1 Introduction

Modern quantitative finance requires the rapid synthesis of both structured data (live market pricing) and unstructured data (earnings reports, macroeconomic analyses). FinAgent addresses this challenge by deploying a multi-agent workflow that autonomously ingests live tickers or CSV batches, retrieves relevant context from a local Vector Database, and formulates defensible investment hypotheses.

2 System Architecture

The FinAgent architecture is designed for high availability, fault tolerance, and modularity. It comprises three primary layers:

- **Data Ingestion Engine:** Handles multi-modal inputs, including live API ticker streams (via Yahoo Finance), simulated data entry, and CSV batch processing.

- **Retrieval-Augmented Generation (RAG):** A privacy-preserving local vector database storing chunked financial reports to ground the LLM in facts.
- **Orchestration & Reasoning:** A deterministic state machine (LangGraph) that routes data between specialized AI agents utilizing the `openai/gpt-oss-120b` model for massive cognitive scale, reasoning, and deterministic JSON generation.

2.1 Multi-Tier Fault Tolerance

A critical architectural feature is the *Fault-Tolerant Feed Engine*. In financial environments, data streams can drop. FinAgent implements a 3-tier failsafe:

1. **Primary:** Attempts rapid live data fetch via WebSocket/Fast Info.
2. **Secondary:** If markets are closed (e.g., weekends) or the primary endpoint fails, the system cascades to a 1-month historical lookback to extract the last known closing price.
3. **Tertiary:** If the API provider rate-limits or blocks the host IP, the system generates a simulated proxy price and injects a `[SYSTEM FALLBACK EVENT]` warning into the observability trace, guaranteeing continuous pipeline execution.

3 Agent Design

To prevent hallucinations and enforce strict task boundaries, the cognitive workload is divided into two distinct, specialized agents operating in sequence.

3.1 The Analyst Agent

The Analyst Agent is the primary reasoning engine powered by `openai/gpt-oss-120b`. Its system prompt restricts it to evaluating the normalized price signal against retrieved RAG contexts.

- **Input:** Asset ticker, live price, and top- k document chunks.
- **Output:** A bullish/bearish/neutral classification, a raw confidence integer, and a natural language hypothesis. If no RAG documents are found, it triggers a *Zero-Shot Fallback*, explicitly logging that it is relying on intrinsic knowledge.

3.2 The Portfolio Synthesizer Engine

The Synthesizer acts as the execution layer. It takes the unstructured hypothesis generated by the Analyst and forces it into a strict JSON schema.

- **Input:** The Analyst's hypothesis and confidence score.
- **Output:** A rigorously typed `DecisionArtifact` containing an exact action (BUY, SELL, HOLD, WATCH), bulleted evidence, and isolated risk flags.

4 LangGraph State Graph

The orchestration between the agents is managed by LangGraph, ensuring that the system state is immutable, traceable, and strictly typed.

4.1 State Schema

The `AgentState` dictionary maintains the global context for every transaction:

- `normalized_signal`: The current asset and price.
- `retrieved_contexts`: The raw text and source metadata.
- `agent_logs`: An append-only list tracking tool calls, fallback events, and inter-agent handoffs.
- `decision_artifact`: The final structured output.

4.2 Execution Flow

The Directed Acyclic Graph (DAG) enforces a linear execution paradigm: **START** → **Analyze Node** → **Synthesize Node** → **END**.

5 RAG Pipeline Details

The system utilizes a dual-model architecture. While `openai/gpt-oss-120b` serves as the generative reasoning engine, a specialized embedding model is used for document vectorization.

- **Document Parsers:** Supports both PDF and TXT formats.
- **Chunking Strategy:** Uses a `RecursiveCharacterTextSplitter` with a chunk size of 500 characters and a 50-character overlap, keeping the context window dense.
- **Embedding Model:** Utilizes `all-MiniLM-L6-v2` via HuggingFace. This model is chosen specifically for rapid, local, CPU-friendly vectorization, ensuring low-latency retrieval.
- **Vector Store:** FAISS is used for in-memory, high-speed similarity search, retrieving the top $k = 2$ most relevant chunks.

6 Observability & Innovation Features

Observability is a core mandate of FinAgent. The system includes several advanced, user-facing interfaces to audit the AI's reasoning:

6.1 Agentic Trace Logs

Every generated artifact includes an explicit log feed that reveals:

1. The exact source attribution (filename) and text preview of the RAG passages used.
2. System fallbacks (e.g., "ZERO-SHOT FALLBACK" or "API Fetch Failed").
3. Specific tool calls and agent hand-offs.

6.2 RAG Debugger Interface

A dedicated debugging tab allows operators to query the FAISS vector database directly, enabling human-in-the-loop validation of embedding quality and chunk relevance.

6.3 Multi-Turn Conversational Chatbot

Users can select any historical decision artifact from a dropdown and converse with the LLM. The system injects the exact historical state (price, original internal hypothesis, and RAG context) into the chat prompt, allowing the user to interrogate the AI's reasoning and challenge its conviction.

ing the massive reasoning scale of the `gpt-oss-120b` open-source model, the platform serves as a highly reliable, enterprise-grade template for quantitative AI orchestration.

7 Sample Output Artefact

Below is a representation of a generated `DecisionArtifact` serialized from the pipeline. The system also supports exporting all historical artefacts into a single JSON file for compliance auditing.

```
{
  "asset": "NVDA",
  "action": "BUY",
  "confidence_level": "High",
  "confidence_score": 88,
  "evidence": [
    "Q1 expansion in compute supply chain.",
    "Data center margins improved by 120 bps.",
    "Trading near mid-range support level."
  ],
  "risk_flags": [
    "Potential regulatory changes.",
    "Macro-economic tightening."
  ],
  "price_evaluated": "$132.40",
  "timestamp": "14:32:05"
}
```

8 Conclusion

FinAgent successfully bridges the gap between unstructured financial documents and actionable intelligence. By enforcing strict observability, dynamic fallback mechanics, and leverag-